

PRODUCT STATUS REPORT

TELITI

Teknologi Evaluasi Lowongan dan Integritas — an AI platform scoring job-ad integrity to protect jobseekers from scams and ghost jobs.

316 tests passing

Rule layer: complete

Text model: not yet trained

Prepared: 1 August 2026
Repository: TELITI (local working copy)
Scope: MVP build, Days 0–1 of the development plan, plus early work on Days 2–4
Reference: MVP_PLAN.md, STATUS.md, docs/PAPER_REVISIONS.md

1What actually works today	Page 2
2What hasn't been done yet	Page 6

Part 1 — What actually works today

You have a **working web application** that takes a job advertisement, runs it through nine deterministic rules in English or Indonesian, and returns a 0–100 score with per-item evidence — but **the AI model does not exist yet**, so the score itself is currently a placeholder.

Put differently: the entire scaffolding of the product is built and tested. The brain is missing.

The four-layer architecture (concept paper §3.1)

	1. Ingest Text cleaning, field extraction, sanitisation against spoofing	WORKING
	2. Text analysis (AI core) Returns a hash-derived placeholder number	STUBBED
	3. Metadata rules All 9 rules implemented, tested, bilingual	WORKING
	4. Scoring + XAI Fusion logic works; sentence-level explanation needs the model	HALF DONE

Two of four layers are complete. The third is fully designed but blocked on external dependencies. That is roughly where you would hope to be at this stage — the missing piece is a training run, not a design problem.

What the rule layer can actually detect

RULE	CATCHES
payment_request_id	"administration fee", "biaya administrasi", "security deposit" — the single strongest scam marker
contact_messaging_only	Chat-only recruitment; Telegram weighted far above WhatsApp (ordinary in Indonesian business)
salary_implausible_vs_umk	Salary vs. regional minimum wage, gated on entry-level signals
risk_phrase_score_id	~175 weighted phrases across 6 categories, both languages
email_free_provider	Gmail/Yahoo vs. corporate domains; disposable providers weighted higher
email_domain_mismatch	Email domain not matching the company named in the ad
email_absent	No application route offered at all

RULE	CATCHES
url_shortener	<code>bit.ly</code> , <code>s.id</code> — Linktree correctly allowlisted
qualification_conflict	"fresh graduate" + "5 years experience", bilingual patterns

On the concept paper's own §3.4 scenario, **five rules fire** with distinct evidence:

payment_request_id	+9.50 pts	"biaya administrasi"
contact_messaging_only	+6.00 pts	"via Telegram"
salary_implausible_vs_umk	+3.48 pts	2.9x UMK for tanpa pengalaman
risk_phrase_score_id	+3.20 pts	"Kuota terbatas", "tanpa pengalaman"
email_free_provider	+2.20 pts	hrd.rekrutmen2024@gmail.com

On a legitimate posting: **0 of 9 fire**. That false-positive guard is what §3.6 of the concept paper requires, and it holds in both languages.

Things that work better than expected

The locale layer

Originally built Indonesian-only — an English scam fired only 2 of 9 rules. Now **5 of 9**, and Indonesian support is a drop-in: place two reference files in `data/reference/` and it activates on restart, with no code change. This substantially de-risked the dataset problem.

The tri-state rule design

Every rule reports one of three states: *fired*, *clean*, or **unassessed**. That third state is the most valuable design decision in the codebase — a rule that could not run never looks like a rule that found nothing. It is what prevented EMSCAD's data redaction from silently poisoning the model, and it is what keeps the English-only path honest.

Performance

6.9 ms on a typical advertisement, 86 ms worst case at maximum input length (down from 794 ms before hardening). There is ample headroom for the transformer and explainability module within the 1-second latency target.

Test coverage

316 tests — not decoration. They caught a bidirectional-text spoofing vector, a bug that silently divided every Indonesian salary by 12, and a false positive where "the company covers training costs" was flagged as a scam demand.

The TF-IDF baseline — a real, usable result

MODEL	PR-AUC	PRECISION	RECALL	FALSE POSITIVES
TF-IDF + LinearSVC	0.8769	0.930	0.767	5 / 1,702

This is a genuine, quotable, working classifier — trained on the EMSCAD dataset, evaluated on a held-out split it never saw during training. It needs no GPU and scores in milliseconds. **If the transformer training is delayed further, this baseline is a legitimate fallback to ship.**

What is honestly lacking right now

The score is meaningless at this moment. `model_probability` is currently a SHA-256 hash of the input text — not a model prediction. Every Integrity Score the system produces today is synthetic. The web UI displays a persistent warning banner, the `/health` endpoint reports `model_loaded: false`, and every generated summary is prefixed `[STUB]` so this cannot be mistaken for a result.

- **No sentence-level explainability yet.** The concept paper promises identifying "which sentence is suspicious." The current stub fakes this with simple keyword matching; real occlusion-based explanation requires a trained model to occlude sentences against.
- **Rule weights are asserted, not measured.** Nine values, ordered by domain reasoning and capped so no single rule can dominate — defensible, but unverified, because verification needs the evaluation dataset.
- **No ghost-job detection.** Correctly scoped to Roadmap Tahap 3 in the concept paper. This should be stated plainly whenever the product is demonstrated.
- **Wage reference tables are unverified.** Both the Indonesian UMK table and the English wage table were compiled from public reporting and should be confirmed against official sources before any publication.

Part 2 — What hasn't been done yet

The two blockers

Blocker A — GPU verification (~15 minutes once available)

PyTorch must be installed with CUDA 12.8 wheels on the RTX 5050 machine, and one real matrix multiplication run to confirm it executes.

Why this specific check matters: the RTX 5050 is a Blackwell-generation GPU (compute capability sm_120). A plain `pip install torch` installs without error and reports `cuda.is_available() == True` — and then fails on the very first real operation, because the default wheels do not contain sm_120 kernels. Checking availability alone would give false confidence.

Blocks: steps 2.1, 2.2, 2.3, 2.5, 3.2, 3.3, 3.4, 4.1, 4.2 — nine steps of the plan.

Blocker B — Evaluation dataset (~200 annotated items)

The held-out evaluation set is the source of the project's headline metric. Without it, the only available number is EMSCAD cross-validation performance, which the concept paper's own §3.2 explicitly states should *not* be the number led with, since EMSCAD does not represent the target use case.

A helper tool (`ml/add_eval_item.py`) exists to make annotation fast: it prompts for each field, enforces independent double-labelling before any discussion (required for computing inter-annotator agreement), and re-validates the file after every entry. English-language items are now acceptable given the locale layer, and are considerably easier to source (FTC advisories, r/scams, Better Business Bureau reports) than Indonesian equivalents.

Blocks: steps 2.2, 3.1 validation, and 4.2.

Remaining steps, in order

STEP	WHAT IT DOES	DEPENDS ON	GATE
2.1	Fine-tune mDistilBERT transformer	GPU	Val PR-AUC ≥ 0.88
2.2	Zero-shot cross-language probe	GPU + dataset	Produces the honest headline number
2.3	Probability calibration (Platt scaling)	2.1	ECE ≤ 0.05
2.5	Wire the real model into the API	2.1	Replaces the stub; contract already frozen
3.1	Validate rule weights against the holdout	Dataset	Harness built; reports honestly either way

STEP	WHAT IT DOES	DEPENDS ON	GATE
3.2	Set risk thresholds from a precision target	2.1	Replaces current placeholder round numbers
3.3	ONNX export + INT8 quantisation	2.1	p95 latency < 400 ms, F1 drop < 1pp
3.4	Sentence-level explainability (occlusion)	2.1	Full analysis < 1 second
4.1 / 4.2	Final integration and evaluation report	All above	Test split touched exactly once
4.4	Tag <code>v0.1-mvp</code>	4.1–4.3	Freeze point

A rule to hold to once the dataset exists: the rule weights in `ml/rule_weights.py` must never be adjusted in response to performance on the holdout set. The moment they are tuned against it, the holdout becomes training data and the headline metric loses its meaning.

Paper revisions — drafted, ready to apply

Five changes have been identified and written up in full, with Indonesian text ready to paste directly into the concept paper. None of this work depends on either blocker and can be completed immediately.

- §3.3** — Fusion cannot be fitted on EMSCAD as originally planned: measurement showed only 0.02% of documents contain a real email address, and rule availability differs by class (74.8% vs. 67.9%), meaning a model trained on it would learn the dataset's redaction pattern rather than genuine fraud signals.
- §3.3** — The bounded-penalty cap should be stated per-rule (0.10) rather than as an aggregate (0.15), since the aggregate figure was sized for three rules and the system now has nine.
- §3.3** — Rule weights are set a priori and validated, not fitted, because fitting would require a second annotated dataset beyond what the timeline allows.
- §3.2** — SMOTE was tested and found not to help (PR-AUC 0.8638 vs. 0.8662 with class weighting alone), so class weighting is used instead.
- §3.2** — A note on EMSCAD's text quality: HTML markup was stripped without inserting separators in the public distribution, fusing words together in 78% of documents. This is corrected before training.

A sixth item — whether §1.1's Indonesian framing needs an explicit scope limitation — is intentionally left open pending which language the evaluation dataset ends up in.

Also unverified

A Docker deployment configuration has been written (single container, one port, frontend and API served together) but has not been build-tested, because the Docker daemon was unavailable in the environment where it was created. It should be run once — `docker compose up --build` — before any live demo.

Overall assessment

Roughly 60% complete by step count — but the remaining 40% is the part the concept paper is actually about. Everything surrounding the model (the API contract, the rule engine, scoring logic, the web interface, deployment configuration, and the evaluation harness) is built, tested, and working. The model itself is one training run away.

The 15-minute GPU verification check is the single highest-leverage action available: it unblocks nine of the remaining steps. Against a 4–8 day project deadline, having that check unscheduled is currently the largest risk to on-time completion.